Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación y Diseño Digital
Carrera de Software

Aplicaciones Distribuidas (ISR-701)

TA-IND-01 — Informe Académico Individual

Análisis crítico de etcd y comparación con Apache ZooKeeper

Calderón Saltos Joseph Alexander

Paralelo: A
Fecha: 12 de noviembre de 2026

Repositorio GitHub

https://github.com/BelindaTZ/TA-IND-01-Informe-Acad-mico-Individual---CGMT/tree/main/Calderon%20Joseph

Resumen

etcd es un almacén distribuido de clave y valor que mantiene datos de configuración y coordinación, como el estado del plano de control de Kubernetes. Este informe examina cómo su API y el consenso Raft permiten acceder a un estado común desde distintos miembros. El análisis aplica las siete transparencias de distribución, CAP y PACELC, y contrasta el diseño de etcd con Apache ZooKeeper. En un clúster local de tres contenedores se escribió una clave, se consultó desde otro miembro y se detuvo el líder. Los dos miembros restantes eligieron un nuevo líder y confirmaron otra escritura. Tras reiniciar el miembro detenido, sus índices de Raft coincidieron con los del resto y la consulta desde ese nodo devolvió el valor escrito durante su ausencia. El resultado muestra que el clúster pudo reanudar las escrituras al conservar una mayoría. Un incidente documentado ilustra un límite de esa recuperación: índices de Raft iguales pueden coexistir con diferencias en los datos persistidos. La comparación sitúa la principal ventaja de etcd en sus lecturas linealizables por defecto y su coste en la coordinación necesaria para sostenerlas.

1. Introducción

Kubernetes necesita conservar un registro común de los recursos que administra y de sus cambios. Los componentes que consultan esa información deben poder trabajar sobre un estado coherente. Kubernetes usa etcd como almacén de los datos del clúster. El servidor de la API accede a ese almacén y los controladores operan a través de la API de Kubernetes [1]. También pueden usar etcd directamente aplicaciones que necesiten guardar configuración o coordinar quién puede modificar un recurso [2].

La escala relevante tiene dos partes. En el laboratorio hay tres votantes y un único equipo físico. En un despliegue mayor, el número de trabajadores de Kubernetes puede crecer sin añadir un votante de etcd por trabajador. Cada votante conserva una copia del estado replicado y las escrituras dependen de una mayoría. Un grupo de cinco votantes puede soportar la pérdida de dos y necesita tres para confirmar cambios. Aumentar el grupo cambia tanto la tolerancia como el coste de coordinación [3]. Se eligió etcd porque permite observar esas consecuencias con un clúster local y comparar sus garantías con las de otro servicio de coordinación.

2. Arquitectura del sistema

La API v3 recibe solicitudes como `Put`, `Range` y `Txn`. Cada miembro dispone de un servidor, un componente Raft y almacenamiento persistente. Raft es el algoritmo de consenso que coordina la elección del líder y el orden de las operaciones en un registro. El almacén de claves aplica las entradas confirmadas y conserva revisiones para consultas y observación de cambios. El registro persistente permite recuperar el estado tras un reinicio [2], [4]. Bora et al. formalizan en mCRL2 el comportamiento de Raft. Su trabajo permite distinguir el modelo de consenso del código que lo implementa [5].

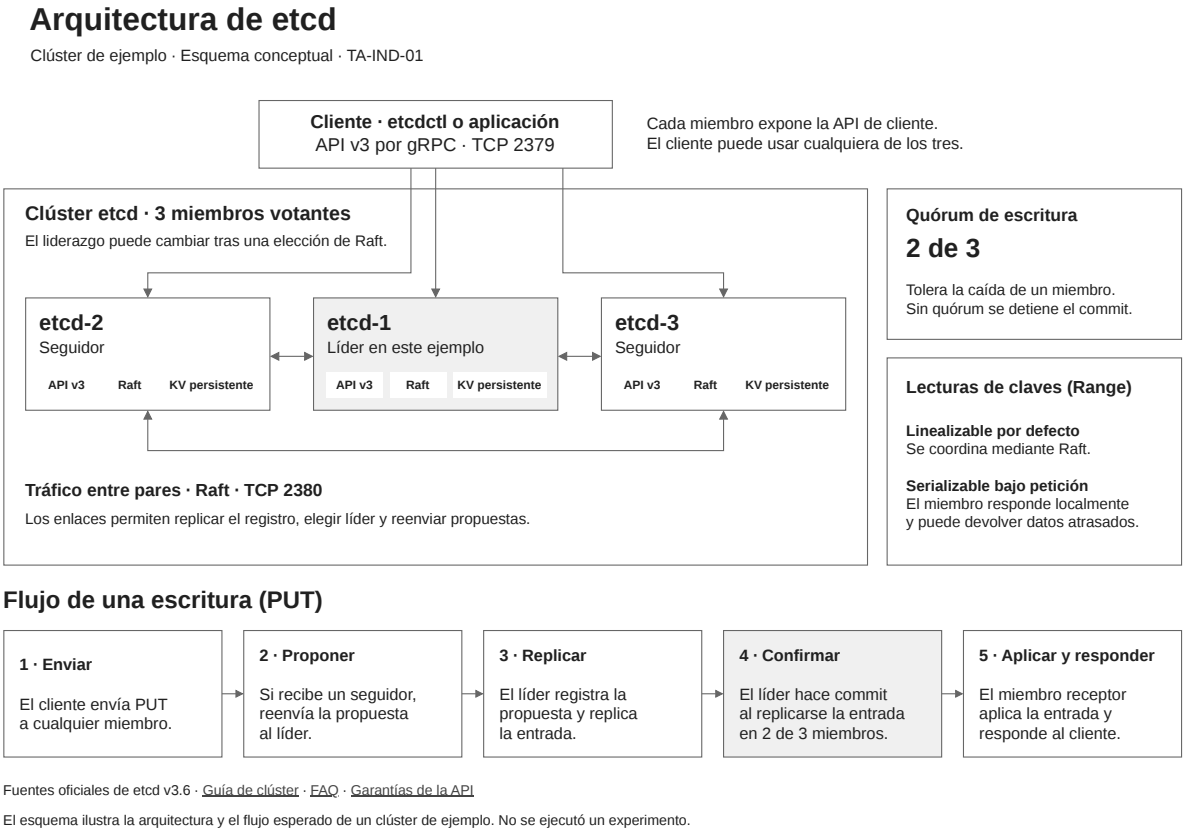


Figura 1: Diagrama del estudiante en draw.io. Los roles corresponden a un estado posible del clúster.

El boceto del Anexo A ya representa al cliente, tres miembros y el paso de una escritura por el líder. El diagrama digital hace explícitos los puertos, distingue el acceso del cliente del tráfico entre pares y muestra dónde interviene el quórum. Su rótulo inferior corresponde a la fase de diseño del esquema. La ejecución posterior se documenta en la sección 4. Los nombres con guion pertenecen al esquema y los nombres sin guion identifican los servicios de Compose.

El cliente puede iniciar una escritura a través de un seguidor. El miembro receptor propone el cambio y el líder dirige la replicación. Con tres votantes, una escritura necesita la confirmación de al menos dos miembros. A esa mayoría se la conoce como quórum. El miembro receptor aplica el cambio confirmado antes de contestar. En nuestro experimento, los clientes se conectaron por el puerto interno 2379 y el tráfico entre miembros usó 2380. Solo `etcd1` publicó un puerto en el equipo anfitrión. Las pruebas usaron los nombres internos de los tres servicios.

3. Análisis con los conceptos de la Unidad 1

3.1. Transparencias de distribución

Las transparencias describen qué aspectos de la distribución quedan ocultos para quien utiliza el sistema [6]. En etcd su alcance depende de la operación y de la configuración del cliente [2], [7], [8], [9].

Acceso. La API ofrece la misma operación y representación de claves en los tres miembros. `etcdctl` usó el mismo `get` al consultar desde otro nodo.

Ubicación. El nombre de una clave no incluye el servidor que la almacena. La transparencia es parcial porque el cliente todavía necesita direcciones alcanzables para conectarse.

Migración. etcd conserva el espacio de claves al reemplazar un miembro mediante una reconfiguración. Mover el servicio o sus datos a otro clúster requiere intervención operativa y puede exigir cambiar los endpoints.

Reubicación. Un cambio de líder conserva el nombre de la clave y la API mientras el sistema está en uso. Una sesión conectada a un miembro que se apaga puede interrumpirse. etcd no traslada esa conexión activa al nuevo líder.

Replicación. El cliente envía una solicitud y Raft coordina las copias. El usuario no selecciona la réplica que debe guardar el cambio. Una lectura local serializable puede revelar retraso entre réplicas.

Concurrencia. Las escrituras confirmadas tienen un orden común. Una transacción puede comparar la revisión de una clave y escribir solo si coincide. La aplicación que hace una lectura seguida de una escritura independiente debe manejar por su cuenta el conflicto entre ambas.

Fallos. La elección y la puesta al día del miembro son automáticas ante un fallo temporal que conserve el quórum. Durante la elección o al perder el endpoint pueden aparecer errores y tiempos de espera.

3.2. CAP y PACELC

En CAP, la consistencia se entiende como una vista linealizable y la disponibilidad exige que cada solicitud a un nodo operativo pueda completarse, incluso durante una partición. En una división de dos miembros contra uno, el lado mayoritario puede elegir líder y confirmar escrituras. El lado aislado no puede confirmar una escritura ni sostener por sí solo la lectura linealizable. Por eso las operaciones fuertes de etcd se clasifican como CP. Una lectura local de un miembro aislado puede devolver una revisión anterior y ofrece otra garantía [6], [7], [9].

PACELC examina además lo que ocurre con la red sana. Una escritura espera la replicación suficiente y una lectura linealizable requiere comprobar que el miembro tiene acceso al estado confirmado bajo un líder válido. En ambas rutas etcd prioriza consistencia, con coste de coordinación. La opción `serializable` permite responder con el estado local y prioriza la latencia de esa lectura a cambio de posible desfase. En PACELC, esto se resume como PC/EC: se mantiene la consistencia durante una partición y también se prioriza cuando la red funciona normalmente, aunque eso cueste tiempo de respuesta. Es una interpretación de las garantías de la API, no una medición de esta práctica. La prueba de este informe no midió latencia ni aisló la red [6], [7].

3.3. Modelos de comunicación

El diseño combina un modelo cliente-servidor en la API con comunicación entre pares dentro del clúster. Los pares pueden votar y cambiar de rol, aunque el líder dirige la replicación en cada término. El cliente de la API v3 usa llamadas gRPC sobre HTTP/2 y TCP. `Put` y `Range` siguen el patrón petición-respuesta. El cliente de línea de comandos espera la respuesta o el vencimiento de su plazo [2], [8].

`Watch` mantiene un flujo de eventos desde revisiones de claves. El cliente puede recibir cambios sin sondear la clave en un bucle. Si se interrumpe el flujo, la revisión conocida sirve para reanudarlo mientras siga disponible en el historial. Entre servidores, los mensajes de Raft transportan votos, latidos y entradas de registro. Se envían a través de conexiones entre miembros. La confirmación de una escritura sigue dependiendo de la mayoría [2], [7], [9].

3.4. Tolerancia a fallos e incidente documentado

Con tres votantes se tolera la indisponibilidad de uno mientras los dos restantes puedan comunicarse. Cuando dejan de llegar latidos del líder y vence el tiempo de elección, un seguidor puede solicitar votos. Con una mayoría se elige otro líder. Al volver, el miembro rezagado puede recibir las entradas faltantes o una instantánea si el registro necesario ya no está disponible. Si se pierden permanentemente dos votantes, se requiere recuperación operativa. Un timeout de cliente deja incierto el resultado de la operación, por lo que repetir una actualización de estado puede requerir una comparación de revisión [7], [9], [10].

En marzo de 2022 se reportó un clúster de tres miembros en etcd 3.5.1 con índices Raft iguales y revisiones distintas. El equipo de etcd vinculó el reporte a un fallo de persistencia al caer un miembro bajo carga [11]. El índice persistente podía avanzar antes de que se confirmaran los datos en el almacén. Una caída entre ambos pasos dejaba un índice que indicaba más progreso del que se había guardado. El problema se introdujo en v3.5.0 y se corrigió en v3.5.3 [12]. El caso distingue el acuerdo sobre el registro de su aplicación durable en cada réplica.

Otro fallo puede manifestarse en el lado cliente. El diseño del cliente documenta reconexiones, selección de endpoints y plazos [8]. Incluso con mayoría, una conexión interrumpida puede devolver un error antes de recuperar acceso. El ensayo local consultó explícitamente los dos endpoints activos después de detener al líder.

4. Experimentación propia

El registro del 12 de noviembre documenta tres contenedores de etcd 3.7.3 en Docker Desktop para Windows 11, cada uno con volumen persistente. El archivo `Experimento/compose.yaml` define los miembros. Al iniciar, Windows rechazó la publicación del puerto 12379. El registro describe cómo se retiró la publicación de `etcd2` y `etcd3` para usar su red interna. Los comandos completos y sus resultados están en `Experimento/comandos_y_resultados.md`. La tabla 1 conserva los datos que permiten interpretar la secuencia.

Cuadro 1: Secuencia observada en el experimento

Paso	Operación	Salida observada
Inicio (fig. 5)	`endpoint status -w table`, tres endpoints	Líder `etcd3`, término 3, índices 11/11.
Escritura y lectura (fig. 6)	`put` por `etcd1`, `get` por `etcd2`	`OK` y `Replicacion-etcd`.
Caída (fig. 7)	`docker compose stop etcd3`, consulta de dos endpoints	Líder `etcd1`, término 4, índices 14/14.
Escritura con dos miembros (fig. 8)	`put` por `etcd2`, `get` por `etcd1`	`OK` y `quorum-2-de-3`.
Recuperación (fig. 9)	`docker compose start etcd3`, estado y `get` por `etcd3`	Líder `etcd1`, término 4, índices 16/16 y valor `quorum-2-de-3`.

El cambio del término 3 al 4 y el reemplazo de `etcd3` por `etcd1` son evidencia de una elección. El segundo `OK`, seguido de una lectura por otro endpoint, muestra que la mayoría confirmó un dato nuevo con el líder inicial detenido. Tras el reinicio, los índices aplicados iguales aportan evidencia de que el miembro reincorporado alcanzó la posición observada en los demás. La lectura desde ese endpoint comprueba que podía servir el valor creado durante su ausencia.

El primer intercambio muestra acceso uniforme y una vista común de la clave. La segunda escritura comprueba progreso con quórum tras una elección. Las capturas son observaciones puntuales. El

intervalo entre ellas incluye tiempo de intervención manual y no sirve para estimar la duración de la elección. `docker compose stop` provoca una detención controlada. Los tres procesos comparten equipo y red virtual. El ensayo cubre la pérdida de un proceso y su retorno, con alcance distinto al de una partición, una pérdida de disco o un fallo del anfitrión.

5. Ejemplos del entorno del estudiante

5.1. Sistema de Gestión Académica de la UTEQ

En el SGA de la UTEQ subo tareas y consulto las actividades de mis asignaturas. Después de realizar una entrega, reviso su estado para comprobar que haya quedado registrada. Aunque acceda nuevamente a la plataforma, espero encontrar la información que guardé anteriormente.

Esta situación se relaciona con la transparencia de acceso y la consistencia de los datos. Como estudiante, me interesa que la entrega quede registrada y pueda consultarla después, sin tener que conocer los servidores que utiliza la plataforma. Sin embargo, esta experiencia no permite determinar cómo almacena o replica la información el SGA.

5.2. WhatsApp

En WhatsApp puedo enviar un mensaje y observar sus estados de envío, entrega y lectura. Estos indicadores no siempre cambian al mismo tiempo, ya que dependen de las condiciones de comunicación y de la disponibilidad del destinatario.

Lo relaciono con la comunicación asíncrona en los sistemas distribuidos. El emisor y el receptor no necesitan estar interactuando al mismo tiempo para intercambiar mensajes. Los indicadores permiten conocer parte del estado de la comunicación, aunque no muestran cómo se procesa internamente cada mensaje.

6. Análisis comparativo con Apache ZooKeeper

La comparación toma las garantías de etcd y las documentadas por Apache ZooKeeper [2], [7], [9], [13], [14].

El punto de decisión es qué debe saber la aplicación al leer. Si un controlador actúa sobre la última configuración confirmada, la lectura linealizable por defecto de etcd encaja con esa necesidad. Si el servicio se organiza alrededor de rutas jerárquicas, sesiones y presencia representada por znodes efímeros, ZooKeeper ofrece esas abstracciones directamente. Su lectura local ordinaria es atractiva para consultas que admiten atraso. En una decisión que dependa del dato más reciente, ese atraso debe tratarse explícitamente. La comparación describe el diseño y las garantías. El laboratorio usó solo etcd.

7. Postura crítica

La ventaja de etcd es que la mayoría y las revisiones ofrecen una base clara para coordinar cambios. En el laboratorio, cambiar de líder y mantener dos miembros bastó para volver a escribir. El coste aparece cuando las escrituras cruzan enlaces lentos o los discos demoran la persistencia. En esos casos, añadir réplicas puede aumentar el coste operativo y no reparte automáticamente el trabajo del líder [3]. Du et al. proponen otra estrategia de elección y Shang et al. estudian una organización de Raft en dos niveles. Yu et al. proponen adaptaciones para redes inalámbricas [15], [16], [17]. Son propuestas de investigación y sus resultados corresponden a sus propios escenarios.

Cuadro 2: Comparación técnica de etcd y Apache ZooKeeper

Criterio	etcd	Apache ZooKeeper
Acceso y datos	Claves en un espacio plano, con rangos por prefijo y revisiones.	Árbol de znodes con rutas y versiones.
Consenso y escritura	Raft replica el registro y confirma con mayoría.	Zab, su protocolo de difusión, ordena los cambios desde el líder y necesita quórum.
CAP y partición	El lado sin mayoría no puede avanzar operaciones fuertes.	El lado sin quórum no confirma escrituras. El modo opcional de solo lectura puede servir datos locales.
Latencia y consistencia	Lectura linealizable por defecto. Lectura local serializable opcional.	Las lecturas ordinarias se sirven desde el servidor conectado y pueden ir retrasadas.
Comunicación	RPC gRPC y flujo `Watch` por revisiones.	Protocolo cliente-servidor sobre TCP y avisos de cambios. Los watches clásicos se disparan una vez.
Concurrencia	`Txn` compara versiones, revisiones o valores.	Actualizaciones condicionadas a una versión y operaciones agrupadas con `multi`.
Fallos y sesión	Reelección, reconexión del cliente y claves asociadas a *leases*.	Reelección, reconexión dentro del plazo de sesión y znodes efímeros ligados a la sesión.

Usaría etcd para configuración y coordinación donde una lectura atrasada pueda causar acciones incompatibles. Para guardar telemetría de alto volumen o para aplicaciones que deban seguir escribiendo en ambos lados de una partición, sus garantías y la dependencia del quórum encajan peor [3], [7]. La decisión depende de qué operaciones toleran atraso y cuáles requieren conocer el último estado confirmado.

Un ensayo de caída y retorno comprueba un recorrido concreto. Gulcan et al. usan modelos para guiar la exploración de estados en implementaciones distribuidas, incluida Etcd-raft [18]. Borgarelli et al. emplean recompensas de exploración para dirigir las pruebas hacia escenarios difíciles de alcanzar [19]. Los dos trabajos explican por qué una elección exitosa en una práctica local deja abiertas preguntas sobre fallos combinados y secuencias de mensajes.

8. Conclusiones

1. La caída del líder inicial cambió el término de 3 a 4. Con dos votantes activos, etcd confirmó una nueva escritura y la lectura por otro miembro devolvió el valor esperado.

2. Al reincorporarse `etcd3`, los tres miembros mostraron índice aplicado 16 y el endpoint recuperado sirvió el valor escrito durante el fallo. La igualdad de índices y la lectura sostienen la puesta al día observada.

3. La comparación con ZooKeeper distingue dos rutas de lectura. etcd ofrece linealizabilidad por defecto y ZooKeeper sirve lecturas ordinarias desde el miembro conectado. La elección afecta qué tan reciente debe ser la información para que la aplicación actúe correctamente.

Declaración de uso de inteligencia artificial

Se utilizó ChatGPT (GPT-6) y ChatGPT Work como apoyo para organizar el informe, preparar borradores en LaTeX, revisar referencias bibliográficas y consultar documentación técnica. La herramienta también ayudó a seleccionar y resaltar fragmentos de tres artículos académicos.

El experimento con etcd se ejecutó en un entorno local con Docker. Los comandos, sus resultados y las capturas originales quedaron registrados en el repositorio. Las fuentes consultadas se contrastaron con sus publicaciones y enlaces oficiales. La revisión final del contenido y la explicación de los resultados corresponden al estudiante.

Referencias

[1] «Kubernetes Components,» Kubernetes, visitado 13 de nov. de 2026. dirección: https://kubernetes.io/docs/concepts/overview/components/

[2] «etcd API,» etcd, visitado 13 de nov. de 2026. dirección: https://etcd.io/docs/v3.7/learning/api/

[3] «FAQ,» etcd, visitado 13 de nov. de 2026. dirección: https://etcd.io/docs/v3.7/faq/

[4] «etcd persistent storage files,» etcd, visitado 13 de nov. de 2026. dirección: https://etcd.io/docs/v3.7/learning/persistent-storage-files/

[5] P. Bora, P. D. Minh y T. A. C. Willemse, «Modelling the Raft Distributed Consensus Protocol in mCRL2,» *Electronic Proceedings in Theoretical Computer Science*, vol. 399, págs. 7-20, 2024, doi: 10.4204/EPTCS.399.4.

[6] M. van Steen y A. S. Tanenbaum, *Distributed Systems*, 4.ª ed. distributed-systems.net, 2023, ISBN: 9789081540636. dirección: https://www.distributed-systems.net/index.php/books/ds4/

[7] «etcd API guarantees,» etcd, visitado 13 de nov. de 2026. dirección: https://etcd.io/docs/v3.7/learning/api_guarantees/

[8] «etcd client design,» etcd, visitado 13 de nov. de 2026. dirección: https://etcd.io/docs/v3.7/learning/design-client/

[9] «Failure modes,» etcd, visitado 13 de nov. de 2026. dirección: https://etcd.io/docs/v3.7/op-guide/failures/

[10] «Disaster recovery,» etcd, visitado 13 de nov. de 2026. dirección: https://etcd.io/docs/v3.7/op-guide/recovery/

[11] liuycsd. «Inconsistent revision and data occurs.» GitHub issue 13766, etcd, visitado 13 de nov. de 2026. dirección: https://github.com/etcd-io/etcd/issues/13766

[12] serathius@. «v3.5 data inconsistency postmortem,» etcd, visitado 13 de nov. de 2026. dirección: https://github.com/etcd-io/etcd/blob/main/Documentation/postmortems/v3.5-data-inconsistency.md

[13] «ZooKeeper Programmer's Guide,» Apache ZooKeeper, visitado 13 de nov. de 2026. dirección: https://zookeeper.apache.org/doc/current/zookeeperProgrammers.html

[14] «ZooKeeper Administrator's Guide,» Apache ZooKeeper, visitado 13 de nov. de 2026. dirección: https://zookeeper.apache.org/doc/current/zookeeperAdmin.html

[15] Z. Du, Z. Qu, Y. Fu, M. Huang y L. Liu, «Multi-strategy-based leader election mechanism for the Raft algorithm,» *Concurrency and Computation: Practice and Experience*, vol. 35, n.º 22, 2023, **artno** e7734, doi: 10.1002/cpe.7734.

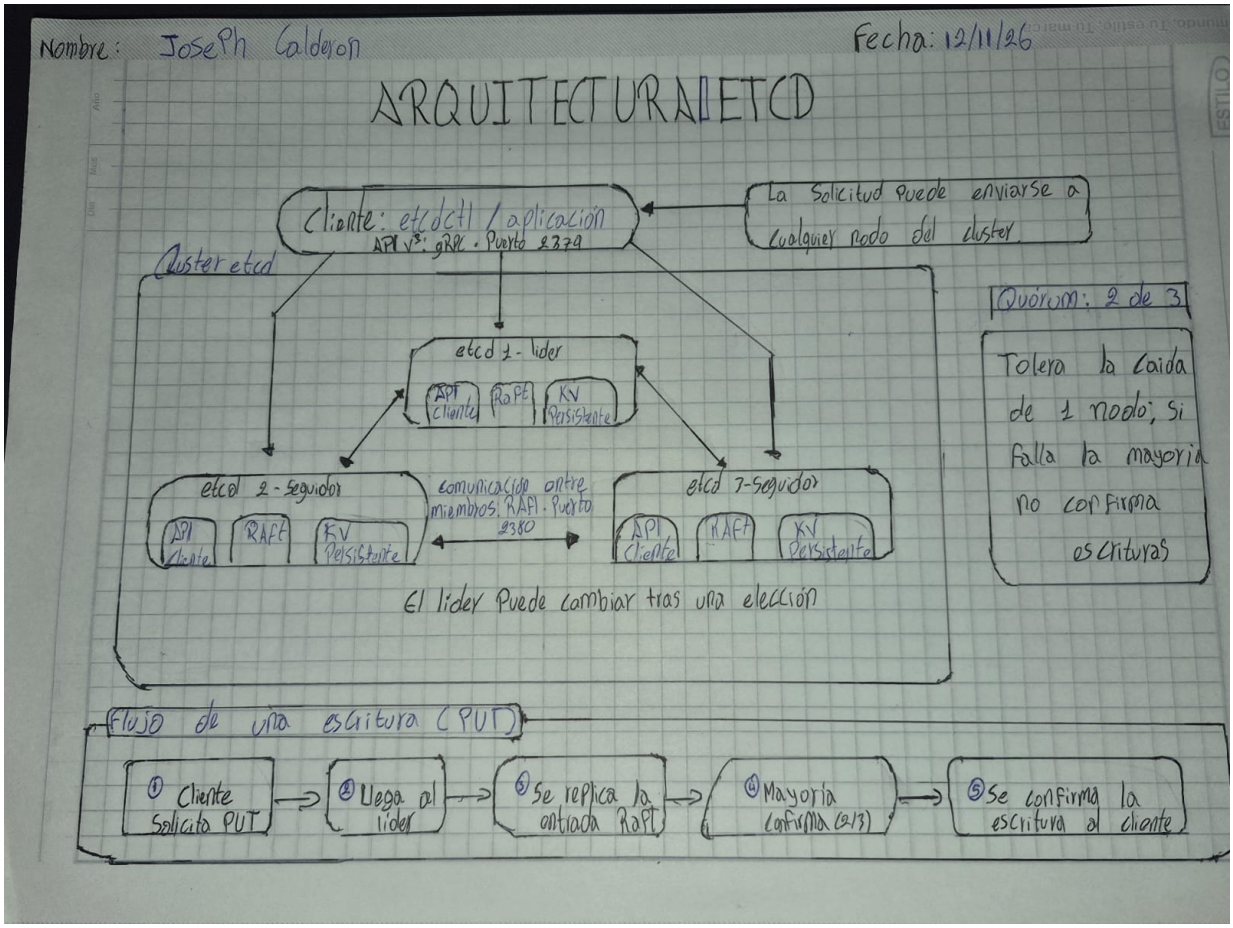
[16] J. Shang, T. Lu, Y. Cai e Y. Li, «DRaft: A double-layer structure for Raft consensus mechanism,» *Journal of Network and Computer Applications*, vol. 236, 2025, **artno** 104111, doi: 10.1016/j.jnca.2025.104111.

[17] D. Yu, H. Wu, Y. Sun, L. Zhang y M. Imran, «Adaptive protocol of raft in wireless network,» *Ad Hoc Networks*, vol. 154, 2024, **artno** 103377, doi: 10.1016/j.adhoc.2023.103377.

[18] E. B. Gulcan, B. K. Ozkan, R. Majumdar y S. Nagendra, «Model-Guided Fuzzing of Distributed Systems,» *Proceedings of the ACM on Programming Languages*, vol. 9, n.º OOPSLA2, 2025, **artno** 282, doi: 10.1145/3763060.

[19] A. Borgarelli, C. Enea, R. Majumdar y S. Nagendra, «Reward Augmentation in Reinforcement Learning for Testing Distributed Systems,» *Proceedings of the ACM on Programming Languages*, vol. 8, n.º OOPSLA2, 2024, **artno** 339, doi: 10.1145/3689779.

A. Bosquejo de arquitectura a mano



B. Capturas de lectura de fuentes

Las páginas provienen de versiones publicadas. Las capturas corresponden a tres artículos citados en el informe. Se incluyen los fragmentos utilizados y sus páginas para relacionarlos con el análisis técnico.

Given the practical significance of the consensus problem and the complexity of the solutions to the problem, found in the literature, formalising and analysing these solutions is highly relevant. The Raft algorithm has been modelled and verified in TLA+ [9] by Ongaro [1], one of the authors proposing the Raft algorithm. This specification contained a couple of minor mistakes which have been fixed, as pointed out by Evrard in [4], where an LNT [5] model of Raft is discussed. An earlier version of the LNT model has been used in the Model Checking contest [7] in 2015, where a few generic requirements were analysed. Another model of the Raft algorithm was presented in [14]. They used the Verdi framework to formally prove the State Machine Safety property, *i.e.*, the property that logs that appear in each node must provide a uniform, consistent view on the state of the servers.

In this paper, we discuss a model written in the mCRL2 language [6] and the formalisation of several properties coined in [12, 11]. The mCRL2 language is a process algebra with data; its process language is based on the algebra of communicating processes (ACP), whereas its data language is based on the theory of abstract data types. The language is supported by the mCRL2 tool set [1], which allows for generating and visualising state spaces, and which can be used to verify properties expressed in the modal μ-calculus with data. While both mCRL2 and LNT are process algebras, their syntax is quite different, and modelling in both languages requires quite a different style. We discuss the design decisions underlying our model of the Raft algorithm, and present modal μ-calculus formalisations of the properties.

Outline. We discuss our mCRL2 model of the Raft algorithm in Section 2. The mCRL2 language is introduced and explained using snippets of our model. For a full explanation of the language, we refer to [6]. In Section 3, we describe the properties that we formalised in the modal μ-calculus. We discuss some of our findings in Section 4, and end with conclusions and future work in Section 5. Full details of the model and the properties can be found in the accompanying artefact in the Mars repository[2].

2 Modelling RAFT in mCRL2

Our mCRL2 model of the Raft Algorithm focusses on the behaviour of the nodes in the network. For our models, we draw inspiration from the TLA+ [11] and LNT [4] specifications of the protocol and, like the LNT and TLA+ specifications, focus on leader election and log replication as these form the core of the protocol. Features such as cluster membership changes and log compaction have not been modelled for simplicity's sake and in the interest of keeping the state space minimal. We additionally model a communication infrastructure that facilitates reliable communication between nodes. Our network model can be modified easily to also capture unreliable communication, but this is not our initial focus. The nodes process commands that can be sent by clients; in our model, the latter is a simple process that has no other purpose than to send commands.

All actors are modelled as dedicated (parameterised) processes in mCRL2: we have `Node` processes, a `Network` process and a `Client` process. The actors run in parallel and can synchronise and exchange data by executing communicating actions. In mCRL2, this is defined by a top-level process such as:

```
init allow( {sendRPC, receiveRPC, clientCommand, advanceCommitIndex, timeout, sendRPCset ...},
            comm ( { sendClientRequest | recvClientRequest -> clientCommand,
                     sendToNetwork | receiveFromServer -> sendRPC,
                     sendToServer | receiveFromNetwork -> receiveRPC,
                     sendToNetworkSet | receiveFromServerSet -> sendRPCset },
```

[1] https://github.com/ongardie/raft.tla
[2] http://mars-workshop.org/repository.html

Anotación de lectura
La elección de líder explica cómo se elige un reemplazo cuando cae el líder de etcd. La replicación del registro permite que los miembros del clúster se pongan al día.

Figura 2: Bora et al., p. 8. Elección y replicación en el modelo de Raft [5].

hand, more detailed criteria, such as traces of messages, may provide too many coverage goals and thus consider each random trace a new behavior, giving up the advantages of coverage-guided exploration.

In this paper, we propose to use *state coverage* in an *abstract formal model* of the system as a coverage criterion and present *model-guided fuzzing* of distributed systems. Abstract formal models are often developed in the design phase of distributed systems to model and formally analyze the underlying protocols [11, 16, 17, 19, 39, 53]. We show that these artifacts are also beneficial in the continuous testing infrastructure of the implementations themselves. Our experiments show that a formal model can serve as a good "guide" for a random testing engine—this is because the formal model often captures the important scenarios of the protocol, and coverage of states in the model correlates well with coverage of interesting behaviors in the implementation.

Our motivation for using abstract model guidance to generate test executions shares common insights with semantic fuzzing [55] and grammar-based fuzzing approaches [30, 40] used for testing sequential programs. Semantic fuzzing aims to cover interesting program executions processing program inputs rather than spending exploration budget for exercising uninteresting, syntactic input parsing logic. While a naive fuzzer is likely to generate inputs that cannot pass the input validation and parsing stage, semantic fuzzing generates test inputs that can go deep into the execution. Similarly, a naive event scheduler for distributed systems is likely to produce tests that spend execution budget in exercising uninteresting, network setup stages. For example, it can explore many different orderings of vote messages during the cluster's leader election phase, barely electing a leader after a prolonged execution. Our approach aims to direct testing toward interesting system behaviors, e.g., processing of user requests once a leader is elected. Similar to grammar-based fuzzing that uses formal specification of the test input to guide test generation, we use an abstract formal model of distributed systems to guide the generation of semantically interesting temporal event schedules.

At a high level, the abstract models recognize semantically interesting behavior; the use of abstract states is a way to provide coverage criteria that capture program semantics. Of course, the use of abstract models is not a panacea: the abstraction may not cover certain implementation details where bugs may lurk. However, lack of structural coverage after model-guided exploration can indicate where additional testing effort should focus, as well as point out aspects of implementation behavior that are not covered by the model.

We have implemented our algorithm for testing distributed systems implementations using TLA+ models [39] of protocols. In a nutshell, our testing algorithm proceeds as follows. We start by exploring random schedules of messages, but feed the same sequence of messages to the TLA+ model. We use the TLC model checker [72] to generate the set of reachable model states corresponding to the explored schedule. We mark a schedule as "interesting" if it covers a new state of the abstract model. We perform, as in coverage-guided fuzzing, a *mutation* of an interesting schedule by swapping the receipt order of two randomly chosen messages or changing the processes to crash. Applying a mutation to an event schedule gives a new schedule to explore that is similar to the original schedule but likely to exercise new system behavior.

We applied our algorithm to test an implementation of the Two-Phase Commit protocol and two industrial implementations of Raft in Etcd-raft and RedisRaft. Our evaluation shows that model-guided fuzzing leads to higher coverage and can detect bugs faster than pure (unguided) random testing, structural code-coverage guided fuzzing, and trace-based coverage-guided fuzzing. Besides reproducing known bugs, we discovered 12 previously unknown bugs in the implementations of Etcd-raft and RedisRaft. More importantly, four of the new bugs could only be detected by our model-guided fuzzing approach.

Anotación de lectura

ModelFuzz busca secuencias de mensajes o fallos que lleven a estados nuevos de Etcd-raft. En mi ensayo probé una sola secuencia de caída y recuperación.

Figura 3: Gulcan et al., p. 282:2. Uso de un modelo abstracto para medir cobertura [18].

In the context of testing, a common approach is to augment the state space with punishments to bias the search to broad exploration [Meng et al. 2023; Mukherjee et al. 2020; Reddy et al. 2020]. For example, Mukherjee et al. [2020] propose an "always punish" strategy: in each step, no matter what action is chosen, the agent is punished by getting a negative reward. Similarly, Meng et al. [2023]; Reddy et al. [2020] assign a zero reward whenever a coverage goal is met for the first time (a "bonus"), and a negative reward otherwise. The negative rewards force the learning algorithm to explore other alternatives, leading to broad exploration and higher coverage. Not providing a punishment when a new coverage goal is met for the first time prioritizes search around that point, akin to coverage-guided fuzzing. These approaches are strongly biased to exploration. However, they do not use any semantic information to guide the search.

Our Contribution. Our work builds on prior approaches but improves the state of the art in reward augmentation in two ways. First, we use insights from the theoretical results in reward-free reinforcement learning [Jin et al. 2020; Zhang et al. 2020] to propose a reward augmentation scheme that, in practice, works surprisingly well in exploring the state space (BonusMaxRL). Like reward-free schemes, we provide a time-decaying bonus: when a coverage goal is reached for the first time, the agent receives a bonus of 1, but the bonus decreases as $1/k$ as the goal is reached k times. This prioritizes new parts of the state space in the short run, but the decay forces the agent to explore other actions if the local search around a newly uncovered goal does not lead to further new goals. However, unlike reward-free approaches, we propagate the bonuses using the *max* function instead of adding them, ignoring bonus values along the path leading to a new state. This focuses the search to target new states over balancing exploration. Therefore, BonusMaxRL behaves differently from prior theoretical results and is tailored for maximizing exploration of unique states within a fixed episode budget.

Second, we augment the exploration using programmer-provided *waypoints*—semantically interesting program states that guide the search to interesting scenarios—which are used to provide additional rewards to the agent (WaypointRL). The waypoints are interesting protocol states that one can get from the description of the protocol; they encode developer insight about interesting "progress points" in the protocol. For example, in a consensus protocol, a waypoint might state that a leader has been elected or that one or more entries have been committed. We show that an RL agent can be biased towards interesting protocol states by providing a sequence of waypoints. In fact, even partial hints that state generic properties of a protocol, but are agnostic to the actual implementation details, are already effective in guiding the search. Moreover, when combined with the coverage bonus of BonusMaxRL, we show that intermediate paths between waypoints are maintained during the learning process, so future episodes can traverse the initial portions of the path to the final waypoint in a directed way. In contrast, if we only provided waypoints, a random search or a punishment strategy does not exploit the waypoints in future episodes.

Empirically, we show that the combination of coverage-based augmentation and waypoints empirically outperforms other approaches in terms of coverage and bug finding. We implement and test three implementations: RedisRaft, Etcd, and RSL. We successfully bias exploration towards 20 out of 26 different waypoints. As a result, we uncover a total of 13 bugs (5 new) in the 3 benchmarks with higher frequency than other approaches in a statistically significant way. Since waypoints allow a developer to bias the search to specific parts of the state space, biased exploration (even when no bugs are found) increases developer confidence in the correctness of specific parts of the codebase.

To apply BonusMaxRL and WaypointRL, we model a distributed system as a Markov Decision Process (MDP). In the process, we explore different models while optimizing for coverage. We define a model that is parametric in the state abstraction and the granularity of actions. We pick a concrete

Anotación de lectura

Al probar etcd, las recompensas pueden guiar la búsqueda hacia un escenario difícil de alcanzar, como elegir un nuevo líder y confirmar una escritura después de varios fallos.

Figura 4: Borgarelli et al., p. 339:3. Recompensa para explorar estados nuevos [19].

C. Capturas del experimento

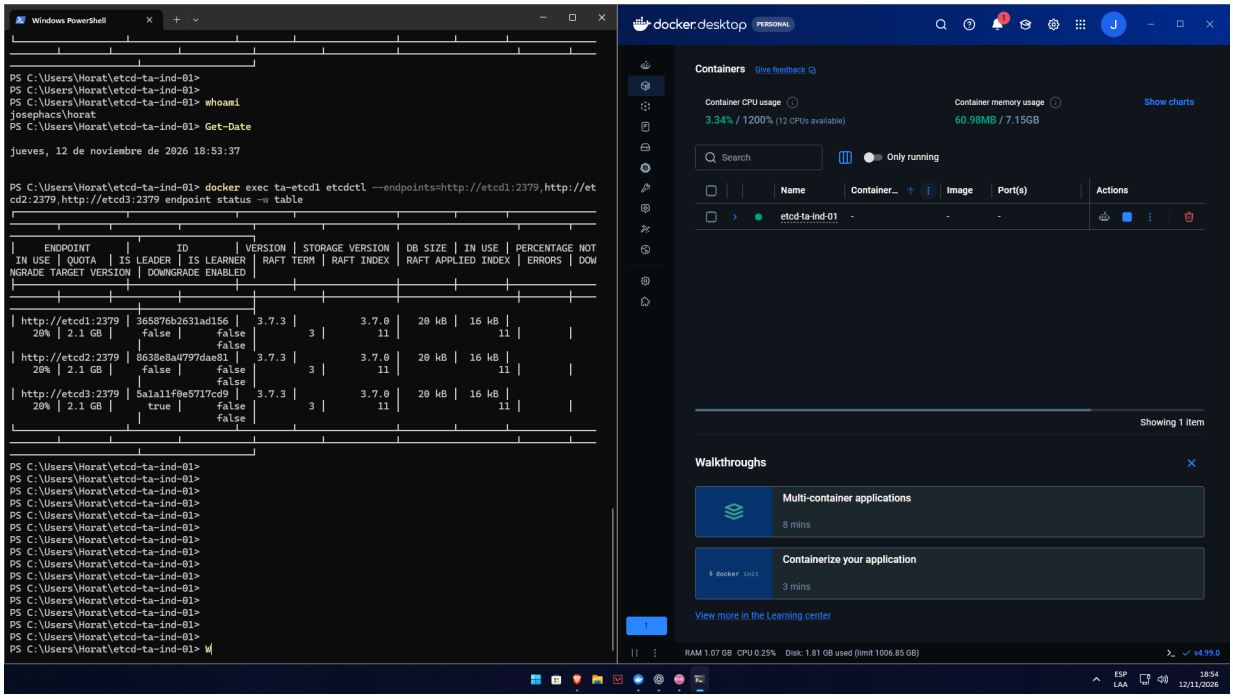


Figura 5: Estado inicial de los tres miembros.

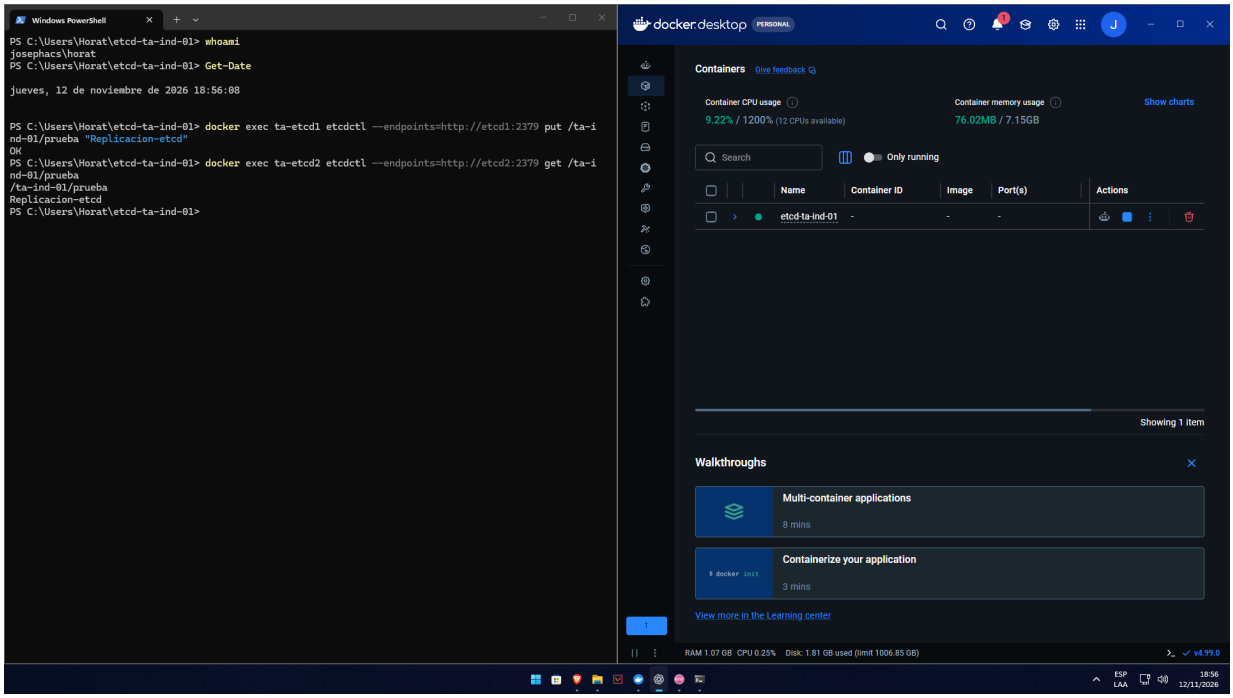


Figura 6: Escritura por etcd1 y lectura por etcd2.

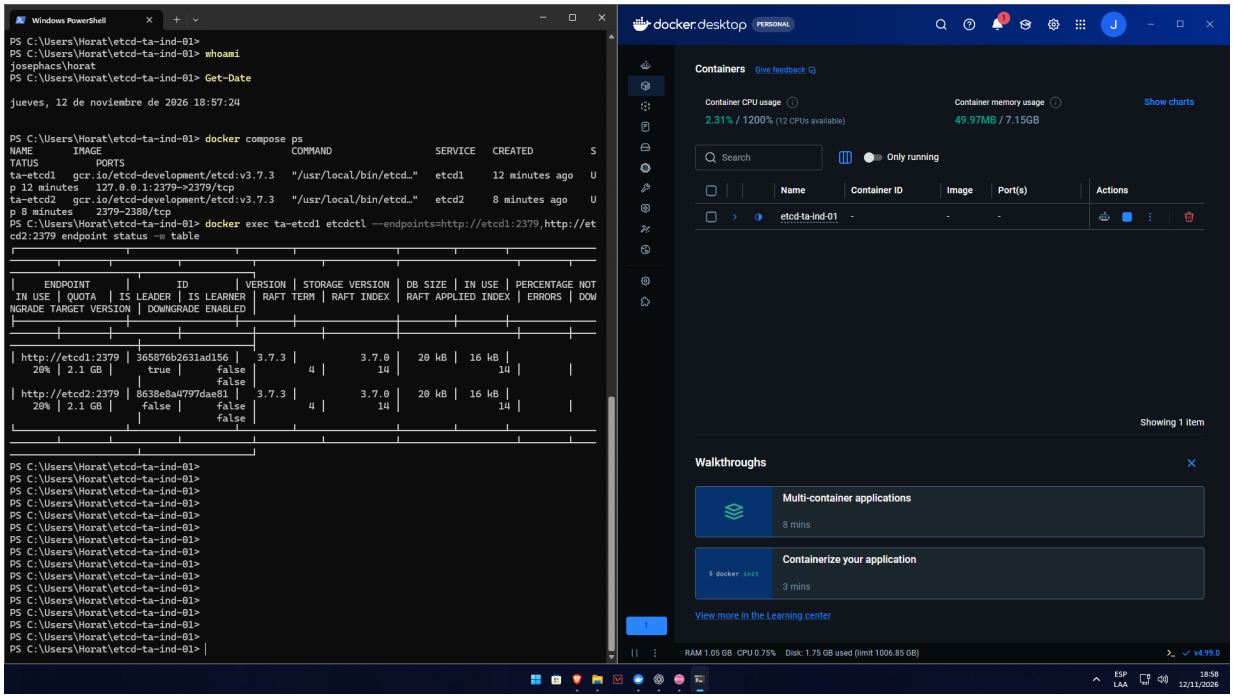


Figura 7: Estado posterior a la detención de etcd3.

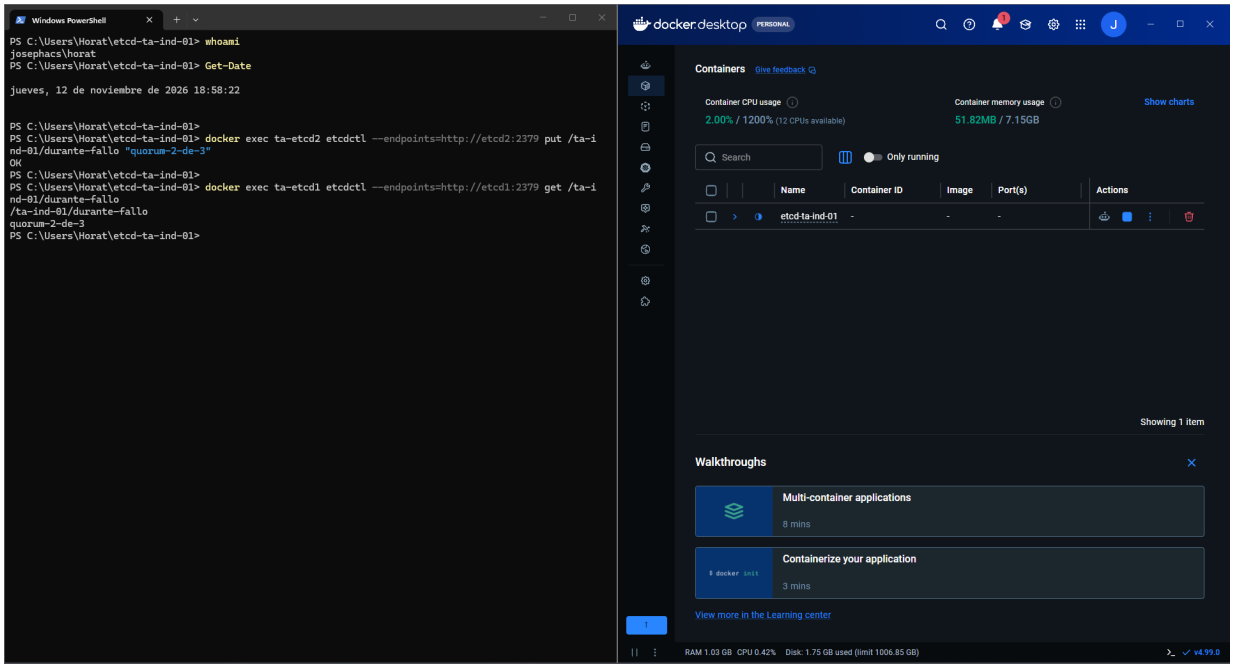


Figura 8: Confirmación y lectura con dos miembros activos.

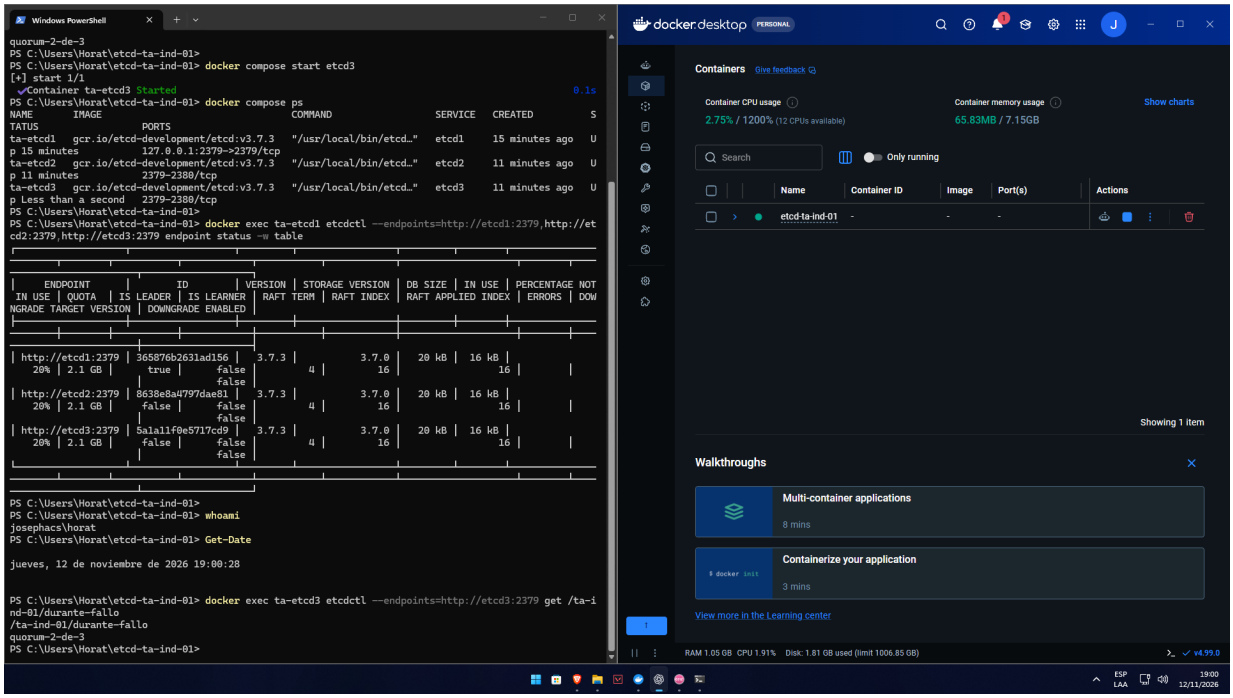


Figura 9: Estado de los tres miembros y lectura desde etcd3 tras el reinicio.